Optimized 256-level Sparse Merkle Tree with Compact Non-Membership Proofs for State Databases

CSPE71 Advanced Cryptography – Mini Project Report

ASIL KHALIFA – Roll No. **106123015**
VII Semester B.Tech (CSE), Academic Year 2026–27
Department of Computer Science and Engineering
National Institute of Technology Tiruchirappalli

Submitted: 28 October 2026

Assigned problem statement. *Program an optimized 256-level Sparse Merkle Tree in C++/Rust generating constant-size O(log N) cryptographic proofs for key non-existence in state databases.*

Abstract

A Sparse Merkle Tree (SMT) over 256-bit keys lets a database commit to its entire key–value state with a single 32-byte root and prove, to anyone who knows that root, both that a key *is* present with a given value and that a key is *absent*. The naive tree has 2^256 leaves and every proof carries 256 sibling hashes (8 KB), which is too large and too slow for state databases that must serve millions of keys. This project implements, in dependency-free C++17, an SMT that keeps the 256-level logical structure but (i) never materialises empty subtrees, (ii) collapses any subtree holding a single key into that leaf, and (iii) transmits only the non-empty siblings of a proof together with a 256-bit presence bitmask. The resulting structure is canonical (the root depends only on the key set), every operation costs O(log N) hash evaluations for N stored keys, and proofs are O(log N) in size with a hard constant upper bound of 8,323 bytes. Non-membership is proven by showing that the queried key's path terminates either in an empty subtree or at a leaf holding a *different* key. Nodes are stored content-addressed, which turns the tree into a versioned state database in which every historical root remains provable until pruned. A command-line tool, 23 unit tests (checked against an independent naive oracle and a Python re-implementation of the specification), and benchmarks up to one million keys are provided. At N = 10^6 a non-membership proof is 710 bytes on average (11.6× smaller than the naive proof), is generated in 15 µs and verified in 26 µs, and the tree sustains about 25,000 inserts per second on a single core.

1. Introduction

Blockchains, certificate-transparency logs, and other verifiable databases need an *authenticated dictionary*: a data structure that commits to a mapping from keys to values with one short digest, such that a prover holding the data can convince a verifier holding only the digest of individual facts about the mapping. Membership facts ("account A has balance 100") are the familiar case. Equally important, and historically harder, are *non-membership* facts: a light client must be convinced that an account does not exist, that a nullifier has not yet been spent, that a certificate has not been revoked, or that a name has not been registered – all without downloading the state.

A **Sparse Merkle Tree** (SMT) answers both kinds of query with a uniform mechanism. It is a complete binary Merkle tree whose leaf index is the key itself: with 256-bit keys the tree has 2^256 leaves, almost all of which are empty. The position of every key is fixed, so "key K is absent" is simply "the leaf at position K is empty", and that can be proven exactly like membership. The price of this elegance, in the naive construction, is that every operation walks 256 levels and every proof carries 256 sibling hashes.

The assigned problem asks for an *optimized* 256-level SMT that produces *constant-size, O(log N)* proofs of key non-existence for state databases. These three requirements are reconciled as follows: the logical tree remains 256 levels deep so that any 256-bit key has a well-defined position; the proof size is bounded by a constant (256 siblings, 8,323 bytes) independent of the number of keys; and, for the N keys actually stored, the expected proof size and the expected cost of every operation are O(log N) rather than O(256).

Contributions of this project

1. A clean specification of a canonical collapsed 256-level SMT with explicit hashing conventions and domain separation, and a proof sketch of why its root is a binding commitment to the key–value set and why non-membership proofs are sound.
2. A C++17 implementation (library + CLI) of the tree over a content-addressed node store that supports versioned roots (snapshots), garbage collection, and single-file persistence.
3. A compact proof encoding with a presence bitmask, binary and JSON serialisations, and an offline verifier.
4. A test suite with a naive reference oracle, tampering tests, adversarial-key tests, and a language-independent cross-check; and an evaluation up to 10^6 keys with charts.

2. Background and Related Work

Merkle trees. A Merkle tree [1] hashes data blocks into leaves and hashes pairs of children into parents up to a root. An inclusion proof for a leaf is its *authentication path*: the sibling at each level, log2(n) hashes for n leaves. Security reduces to collision resistance of the hash function.

Sparse Merkle trees. Laurie and Kasper [2] proposed the SMT for *revocation transparency*: a tree with 2^256 leaves indexed by the hash of the item, where a leaf is a fixed value if the item is absent and a different value if present. Because almost all subtrees are empty and all empty subtrees of the same height have the same hash, the 257 "default hashes" can be precomputed, and a tree with N items needs only O(N log N) real nodes. Dahlberg, Pulls and Peeters [3] studied *efficient* SMTs and the trade-offs between caching strategies for the default hashes; Bauer's thesis on sparse Merkle trees [4] and the Ethereum research community formalised the idea that a subtree containing one key can be represented by that leaf alone, which shortens update and proof cost to O(log N). This "collapsed" variant is used in production by Celestia's `smt` library [5], the Aergo state trie, and in a

16-ary form by Diem/Aptos's Jellyfish Merkle Tree [6]. Ethereum's Merkle–Patricia trie [7] solves the same problem with path compression but at the cost of a more complex node format and non-constant proof structure.

Domain separation. RFC 6962 (Certificate Transparency) [8] hashes leaves with a 0x00 prefix and internal nodes with a 0x01 prefix so that no leaf can be reinterpreted as an internal node. Omitting this is a classic mistake (e.g. Bitcoin's CVE-2012-2459 duplicate-transaction root ambiguity), so this project follows RFC 6962 exactly.

State databases. Blockchain nodes keep their state (accounts, storage slots, UTXO sets) in an authenticated dictionary whose root is placed in every block header. Clients then verify responses against that root. Because a chain has many historical roots, the backing store is usually *content-addressed* (a node is stored under its own hash in LevelDB/RocksDB) so that versions share unchanged subtrees. This project adopts the same storage model.

3. Design

3.1 Hashing conventions

All hashes are SHA-256 (a self-contained FIPS 180-4 implementation is included; the hash is isolated in one module and can be swapped). The conventions below define the tree completely; a Python re-implementation written from this table alone (`tests/reference_check.py`) reproduces the C++ roots bit for bit.

Object	Definition
Key path K (256 bits)	`SHA-256(user key)`; a raw 32-byte path can also be supplied
Value hash V	`SHA-256(value bytes)`
Leaf hash	`SHA-256(0x00 || K || V)`
Internal node hash	`SHA-256(0x01 || left || right)`
Empty subtree (any height)	the fixed placeholder `0x00…00` (32 zero bytes)
Bit *i* of a path	most-significant bit first: bit 0 is the MSB of byte 0; 0 = left, 1 = right

Hashing the user key to obtain the path serves two purposes: it makes keys of arbitrary length addressable, and it prevents an adversary from choosing keys with long common prefixes to inflate tree depth (Section 4.4).

3.2 Structure and the two collapse rules

Logically the tree is a complete binary tree of depth 256; key `K` lives at the leaf reached by following its bits from the root. Two rules turn this into a structure with O(N) nodes:

1. **Empty-subtree rule.** A subtree containing no keys is represented by the placeholder hash and is never stored, whatever its height.
2. **Single-leaf rule.** A subtree containing exactly one key `(K, V)` is represented by the leaf hash `SHA-256(0x00 || K || V)` alone, whatever its height. In other words a leaf "floats up" to the highest position at which it is alone.

Equivalently, the canonical hash of a set `S` of (key, value-hash) pairs rooted at depth `d` is

```
root(S, d) = placeholder                                  if |S| = 0
           = H(0x00 || K || V)                            if S = {(K, V)}
           = H(0x01 || root(S0, d+1) || root(S1, d+1))    otherwise,
```

```
where S0 / S1 are the pairs whose bit d is 0 / 1
```

This definition is a pure function of the set S, so the **root is canonical**: it is independent of the order in which keys were inserted, updated or deleted. The incremental algorithms in Section 3.3 maintain exactly this structure, which the test suite verifies by comparing their output with a direct recursive evaluation of the formula after random sequences of operations.

A consequence of the single-leaf rule is that a leaf sits at depth `1 + LCP(K)`, where `LCP(K)` is the length of the longest prefix K shares with any other stored key. For N uniformly random paths the expected maximum depth is about log2 N + O(1) and the average depth is log2 N + 1.3 in our measurements. Internal nodes can have one empty child (this happens whenever the two keys below share more prefix bits); such nodes form the short "chains" that would be 256 long in the naive tree.

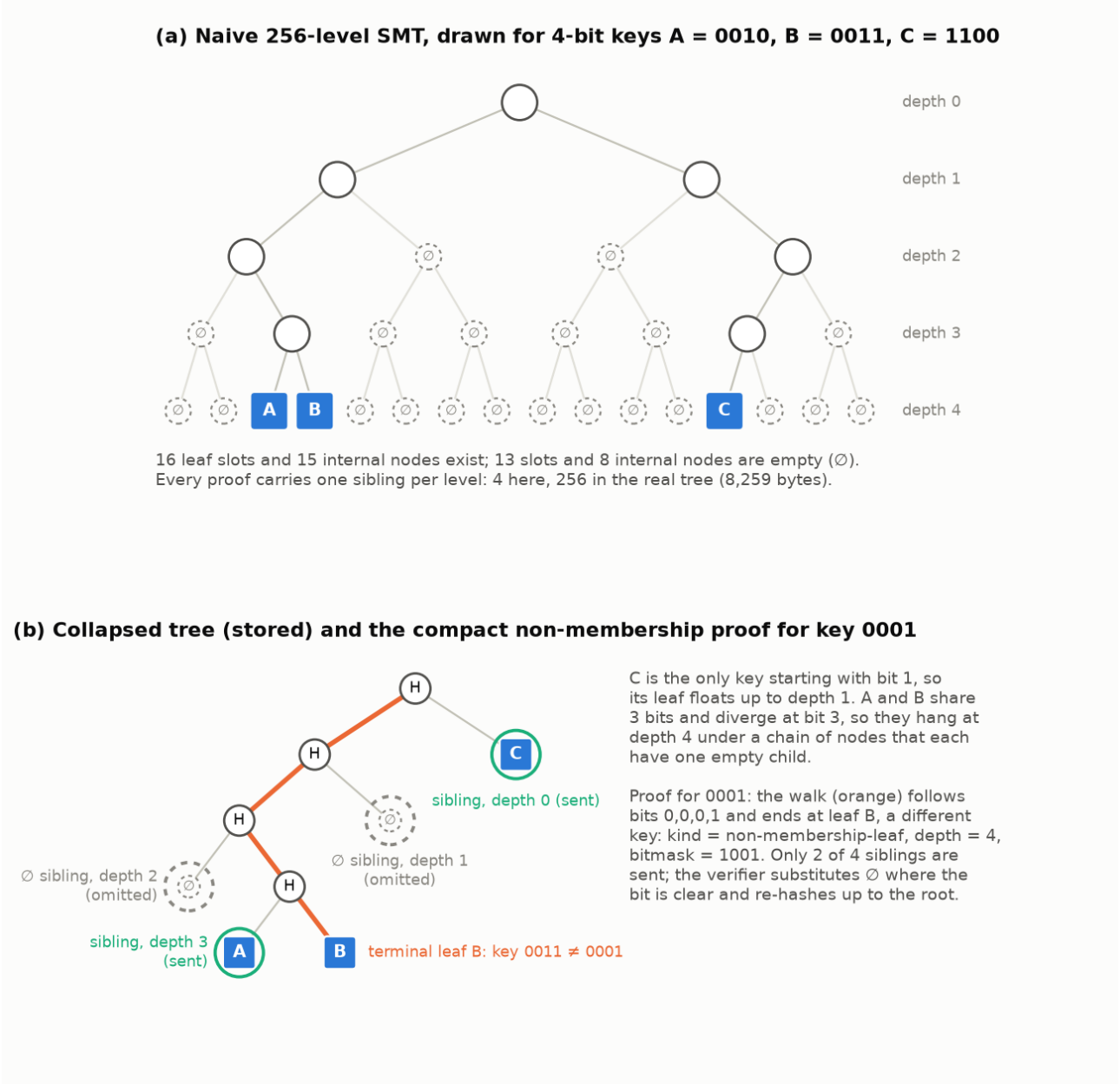


Figure 1. Logical versus stored structure. The naive tree hashes 256 levels for every key; the collapsed tree stores only non-empty internal nodes and floats single leaves up.

3.3 Operations

Lookup. Walk from the root following the key's bits. Stop at the placeholder (absent), at a leaf (present if and only if its key equals the query), or after 256 levels. Cost: the depth of the terminating

node, O(log N) expected.

Insert / update (`put`). Recursively rebuild the path of `K`:

- reaching an *empty* subtree: the new leaf takes its place;
- reaching a *leaf with the same key*: it is replaced;
- reaching a *leaf with a different key* `K'`: compute `d = LCP(K, K')`; create an internal node at depth `d` with the two leaves as children, and a chain of internal nodes from depth `d−1` up to the current depth, each with one empty child (*split*);
- reaching an *internal node*: recurse into the child selected by the key's bit, then rebuild the parent from the new child and the untouched sibling.

Delete (`remove`). Same recursion with the "new leaf" being empty; when a parent is rebuilt from an empty child and a leaf sibling, the leaf is returned in place of the parent, so the surviving leaf floats back up as far as it can (*collapse*). This exactly inverts a split, which is what keeps the structure canonical.

Every rebuilt node is a new content-addressed node; nodes of the old path are released unless history is being kept. Cost: `depth + (d − depth)` hashes for a split, i.e. O(log N) expected (20.8 hashes per insert at N = 10^6, measured).

Hint optimisation. Each stored internal node records whether each child is a leaf (two flag bits that are not part of the hash). During an update the kind of a child is therefore known without fetching it, halving the number of store lookups per level.

3.4 Proofs and the compact encoding

A proof for key `K` against root `R` is produced by the same walk as a lookup, collecting at each level the sibling of the node on the path. The walk ends in one of three ways, giving three proof kinds:

Kind	Terminal node	What the verifier recomputes
`membership`	leaf with key K and value hash V	`H(0x00 || K || V)`
`non-membership-empty`	the placeholder	`0x00…00`
`non-membership-leaf`	leaf with a *different* key `K'` and value hash `V'`	`H(0x00 || K' || V')`

Compaction. Most siblings near the bottom of the path are empty subtrees whose hash is the public constant placeholder; sending them is pointless. The proof therefore carries a 256-bit *bitmask* in which bit *i* is set if and only if the sibling at depth *i* is non-empty, and the list of non-empty siblings in root-to-leaf order. The verifier substitutes the placeholder wherever the bit is clear.

Verification (`verify_proof`):

```
input: proof (kind, depth, bitmask, siblings[], V, K'), trusted root R, queried key K
1. check depth <= 256, no bitmask bit set at index >= depth,
   popcount(bitmask) == |siblings|
2. cur := terminal hash according to kind (table above);
   for non-membership-leaf additionally require K' != K and
   LCP(K', K) >= depth           (the other leaf really lies on K's path)
3. for i = depth-1 down to 0:
       sib := bitmask[i] ? siblings.pop_back() : placeholder
       cur := bit_i(K) == 0 ? H(0x01 || cur || sib) : H(0x01 || sib || cur)
4. accept iff cur == R
```

The verifier uses the *queried* key's bits for the left/right decisions, so a proof is bound to the key it was asked for. For `membership` the verifier also checks `SHA-256(value) == V`.

Size. The compact binary encoding is `1 (kind) + 2 (depth) + 32 (key) + {0 | 32 | 64} (terminal data) + ceil(depth/8) (bitmask) + 32·s` bytes, where `s` is the number of non-empty siblings. Since `depth ≤ 256` and `s ≤ depth`, the size is at most 1 + 2 + 32 + 64 + 32 + 8192 = **8,323 bytes** – the constant bound. For N random keys `s ≈ log2 N + 0.3` (measured), so the expected size is about 32·log2 N + 100 bytes, i.e. **O(log N)**: 393 bytes at N = 10^3 and 710 bytes at N = 10^6 for non-membership proofs. Crucially, compaction decouples proof *size* from tree *depth*: even on adversarial keys that push leaves to depth 211, only ~10 siblings are non-empty, and the proof stays near 420 bytes (Section 6.4).

The proof also has a JSON form (used by the CLI) that carries the same fields plus the root it was generated for and the original user key for readability.

3.5 Storage model: a versioned state database

Nodes are stored in a `NodeStore` keyed by their own hash (content-addressed). The provided backend is an in-memory hash map; the interface is four calls (`find`, `put`, `erase`, `for_each`) so that a LevelDB/RocksDB backend can be substituted without touching the tree logic. Content addressing yields:

- **Versioning.** Every `put`/`remove` produces a new root while leaving old nodes intact; any historical root remains fully queryable and provable (`prove_at(store, old_root, key)`), exactly as a blockchain node serves proofs for past blocks. The CLI records every committed root in a history list.
- **Garbage collection.** With `keep_history = false` the nodes replaced along the updated path are released immediately (they are referenced only by the previous version), so the store holds exactly the live nodes of the current tree – a property asserted after every operation in the tests. With history on, `prune(roots)` performs a mark-and-sweep from the roots to keep and deletes everything else.
- **Persistence.** The CLI serialises the root history and the node set to a single file (`smt.db`); the format is a flat list of `(hash, kind, flags, 64 bytes, [value])` records.

Leaf nodes carry the raw value bytes as an attachment (the hash commits only to `SHA-256(value)`), so a lookup returns the value and a membership proof can be checked against the actual value.

4. Security Analysis

Let `H` be SHA-256, modelled as collision resistant (and hence second-preimage resistant).

4.1 The root is a binding commitment

By the canonical definition, two different key–value sets produce different roots unless a collision of `H` is found: the recursive structure is a Merkle tree, and a standard induction on depth shows that any two distinct inputs that hash to the same node yield a collision at some level. The domain-separation prefixes are essential here: without them a 64-byte "value" could be chosen so that a leaf hash equals an internal node hash, letting an attacker present an internal node as a leaf (or vice versa) and forge a proof for a key that was never inserted. The 0x00/0x01 prefix means the two preimages differ in their first byte, so no such confusion is possible.

4.2 Soundness of non-membership proofs

Claim: an adversary who knows the tree cannot produce a `non-membership` proof for a key `K` that is present (and vice versa) without breaking `H`. Consider the verifier's recomputation. It starts from a

terminal hash `t` and hashes upward using `K`'s bits; acceptance means the recomputed value equals `R`. Because `R` commits to a unique canonical tree, the only way to reach `R` without a collision is to supply, at each depth, exactly the hashes of the real tree along `K`'s path, ending at the real node at depth `depth` on that path. Three cases:

- `non-membership-empty`**.** The verifier starts from the placeholder, so the adversary must claim that the real node on `K`'s path at some depth is empty. If `K` is present, every node on its path is non-empty, and a non-empty node's hash is a SHA-256 output; equating it to the all-zero placeholder is a preimage of zero, which is infeasible.
- `non-membership-leaf`**.** The verifier itself recomputes `t = H(0x00 || K' || V')` from the proof's `K'`, `V'` and rejects if `K' = K`. For acceptance the real node on `K`'s path must be exactly the leaf `(K', V')` with `K' ≠ K`. If `K` were present, the leaf on its path would carry key `K` (the single-leaf rule places at most one leaf on any root-to-leaf path), so no valid `K'` exists.
- **Forging membership of an absent key** requires a leaf `H(0x00 || K || V)` on `K`'s path, which the real tree does not contain.

In addition the verifier checks that `K'` actually shares the first `depth` bits with `K` (defence in depth: a malformed proof is rejected before any hashing), that the bitmask has no bits beyond `depth`, and that the sibling count equals the bitmask's population count, so every byte of a proof is accounted for. The test `tampered_proofs_are_rejected` exercises each of these paths, including relabelling a membership proof as non-membership, swapping `K'` for `K`, flipping sibling bits, truncating the sibling list, and shifting the depth.

4.3 Canonicality prevents equivocation

Because the structure is a function of the key set, a prover cannot maintain two differently shaped trees for the same data and choose whichever is convenient; nor can two honest nodes that applied the same updates in different orders disagree on the root. The test `root_is_independent_of_insertion_order` shuffles 500 insertions five times and interleaves extra insert/delete pairs, always obtaining the same root.

4.4 Adversarial keys and denial of service

Operation cost and the verifier's hashing cost are proportional to leaf depth, which is `1 + LCP` with the nearest key. If users could pick raw paths, an attacker could insert keys sharing a 250-bit prefix, forcing ~250 hashes per operation. Deriving the path as `SHA-256(user key)` makes a `p`-bit shared prefix cost about 2^p hash evaluations to find, so in practice depths stay near log2 N. Even in the worst case the data structure remains correct, and – thanks to compaction – proof *sizes* remain small; only CPU time degrades, and it is bounded by 256 hashes (~0.2 ms) per operation. Section 6.4 measures this case directly.

4.5 What the proofs reveal

A `non-membership-leaf` proof discloses the path and value hash of one neighbouring key, and every proof discloses sibling hashes. This is inherent to Merkle proofs and acceptable for public state databases (paths are already hashes). Hiding neighbours would require zero-knowledge membership proofs, which are outside the scope of this project.

5. Implementation

The implementation is about 2,700 lines of C++17 (library, CLI, tests and benchmark) with no third-party dependencies (not even OpenSSL; SHA-256 is implemented in-tree and checked against

the FIPS test vectors). It builds with CMake or plain `make`.

Module	Purpose
`sha256.hpp/.cpp`	FIPS 180-4 SHA-256, streaming interface
`hash.hpp/.cpp`	leaf/node/key/value hashing, placeholder, bit and hex utilities, hash-call counter
`node_store.hpp`	`Node` record, `NodeStore` interface, in-memory backend
`smt.hpp/.cpp`	`SparseMerkleTree`: `put`, `remove`, `get`, `prove`, `prove_at`, `get_at`, `prune`, `stats`, `for_each_leaf`
`proof.hpp/.cpp`	`Proof` record, compact binary and JSON encodings, `verify_proof`, `verify_membership`, `verify_non_membership`
`db_file.hpp/.cpp`	single-file persistence of root history + node set
`json.hpp/.cpp`	minimal JSON reader/writer for proof files
`cli/main.cpp`	the `smt` command-line state database
`tests/`	23 unit tests, naive reference oracle (`reference_smt.hpp`), Python cross-check
`bench/`	benchmark harness and chart script

The `smt` CLI exposes the tree as a small versioned state database: `init`, `put`, `del`, `get`, `batch`, `root`, `history`, `prove` (membership or non-membership, to a JSON file), `verify` (offline – only the proof file and a trusted root are needed), `dump`, `stats`, and `prune`. Exit codes distinguish *valid*, *invalid*, and *not found* so the tool can be scripted. A complete transcript is included as `evaluation/sample_cli_session.txt`; an excerpt:

```
$ smt --db demo.db prove mallory --out proof_mallory_nonmembership.json
proof type: non-membership-leaf
depth: 2, non-empty siblings: 2, size: 164 bytes (uncompressed 256-level proof would be 8259 bytes)

$ smt --db demo.db verify proof_mallory_nonmembership.json
proof type : non-membership-leaf
key        : c0a497761b175379ed63397cc980546559faa84ca9cbeede773117c31508b6ac
root       : 5be5ad11a7a5732d9e6526b5df025eab306219e95dadbb40f95d2a0f359fe396 (from database)
depth      : 2, siblings: 2, size: 164 bytes
result     : VALID -- key is NOT present in the tree

$ smt --db demo.db verify proof_tampered.json        # one hex digit of a sibling flipped
result     : INVALID -- computed root does not match the expected root
```

Library usage is three lines: construct a store and a tree, `put`/`remove` keys, call `prove` and one of the `verify_*` functions.

6. Testing and Evaluation

6.1 Correctness testing

The 23 tests (`evaluation/test_output.txt`, all passing) cover:

- **SHA-256**: FIPS vectors ("abc", the 448-bit and one-million-'a' messages), padding boundaries at 55/56/64 bytes, streaming versus one-shot.
- **Reference oracle**: after random inserts, overwrites and deletions of 2–1,000 keys the incremental root equals a from-scratch evaluation of the canonical formula; the store holds exactly the live nodes; deleting everything returns the empty root with zero nodes.
- **Canonicality**: insertion-order independence and insert/delete reversibility.

- **Proofs**: membership proofs for every key of a 2,000-key tree and non-membership proofs for 2,000 random absent keys (both terminal kinds occur) verify; the same proofs fail against a different root or key.
- **Tampering**: nine distinct manipulations of a valid proof are rejected (Section 4.2).
- **Adversarial keys**: 64 keys sharing a 250-bit prefix produce depth 256; all proofs verify and respect the 8,323-byte bound.
- **Versioning**: historical roots stay provable; after `prune` the old versions are gone and the current one intact; a 50-version chain is checked exhaustively.
- **Serialisation and persistence**: binary and JSON round-trips, truncation detection, save/load of a 200-version database.
- **Regression vector**: the root of `{alice→100, bob→250, carol→7}` is frozen to `973afc72…ad8f4c` and independently reproduced by `tests/reference_check.py` using only Python's `hashlib`.

6.2 Benchmark setup

Single thread, in-memory store, history off, GCC 13.3 `-O3`, WSL2 Linux 6.6 on an Intel Core Ultra 9 285H (one core used; see `evaluation/bench_output.txt`). Keys and 32-byte values are pseudo-random (seed 20261028). For each N the harness inserts N keys and then measures 10,000 lookups, membership proofs (+ verification), non-membership proofs for random absent keys (+ verification), overwrites and deletions. Proof sizes are the compact binary size of Section 3.4. The naive 256-sibling membership proof is 8,259 bytes. Run `./build/smt_bench --csv out.csv` to reproduce; the raw numbers are in `evaluation/bench_results.csv`.

6.3 Results

N	insert µs	hashes / insert	get µs	prove non-mem. µs	verify non-mem. µs	non-mem. proof bytes avg (max)	siblings avg	leaf depth avg (max)	RSS MB
1,000	7.8	10.8	0.38	0.65	4.9	393 (517)	9.6	11.3 (19)	4.6
10,000	10.5	14.2	0.56	1.03	6.9	496 (646)	12.9	14.6 (25)	15.8
100,000	18.7	17.5	4.2	2.8	8.4	603 (742)	16.2	17.9 (31)	65.1
1,000,000	40.5	20.8	26.1	15.1	26.3	710 (839)	19.6	21.3 (42)	528.8

Membership proofs behave identically (398 → 719 bytes average, 10.3 → 20.3 siblings). Overwrites cost 8–97 µs and deletions 5–155 µs across the same range; SHA-256 of a 65-byte node input costs 0.48 µs (2.1 M hashes/s) on this machine.

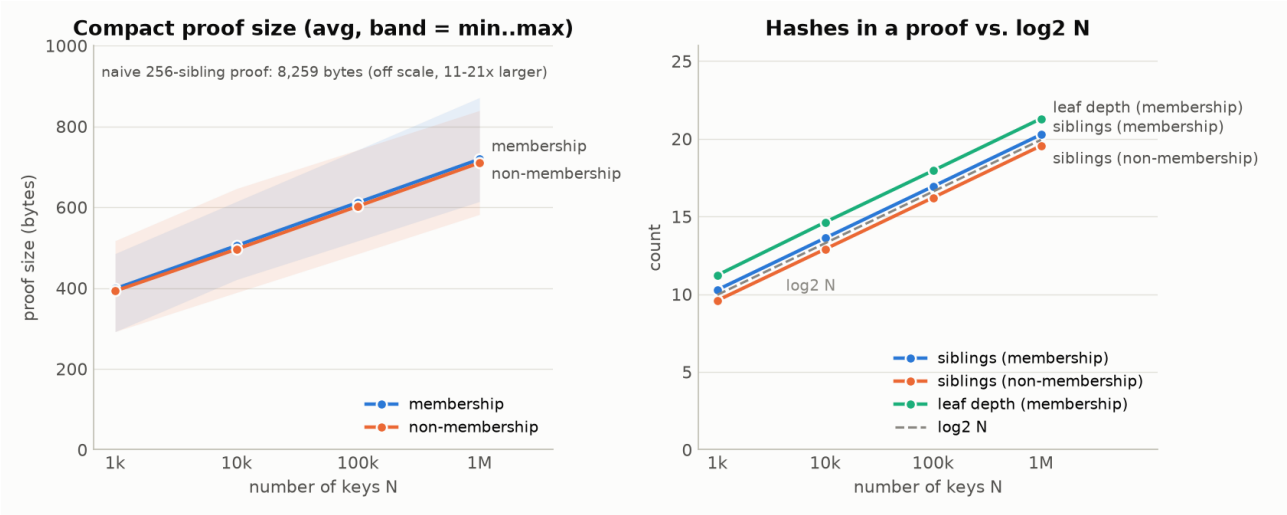


Figure 2. Left: average compact proof size (band = min…max) versus N; the naive proof is 8,259 bytes. Right: non-empty siblings per proof and leaf depth track log2 N.

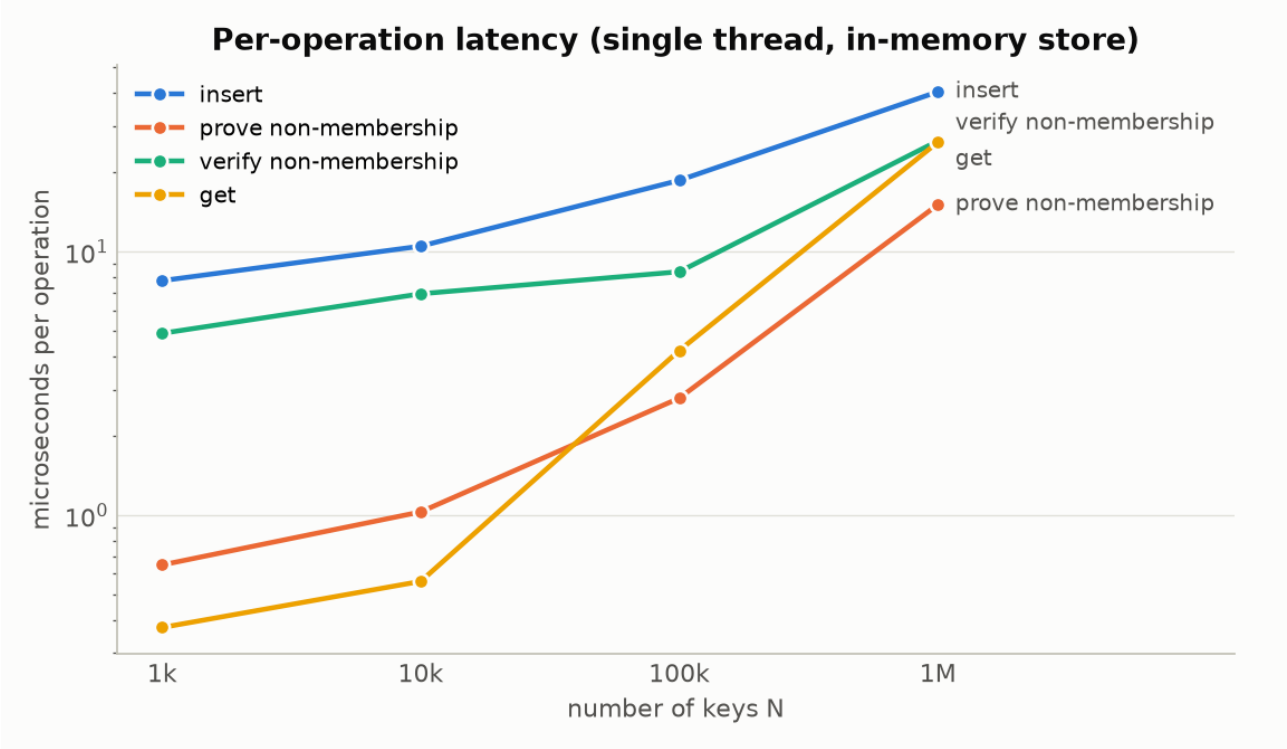


Figure 3. Per-operation latency versus N (log–log). Lookups and proof generation are pure pointer walks; verification and insertion are dominated by SHA-256 calls (≈ 0.5 µs each) and, beyond the cache, by memory latency.

Discussion.

- *Proof size is O(log N) and far below the constant bound.* The number of non-empty siblings is log2 N + 0.3 on average and leaf depth is log2 N + 1.3; each additional decade of keys adds ~3.3 siblings (106 bytes). Compared with the naive proof, compaction gives 21× (N = 10^3) to 11.6× (N = 10^6) smaller proofs, and the largest proof observed at one million keys was 871 bytes, about 10% of the 8,323-byte bound.
- *Hash work is O(log N).* An insert performs 10.8 hashes at N = 10^3 and 20.8 at N = 10^6, versus 256 in the naive tree; a verification performs `depth` hashes (≈ 21 at 10^6).

- *Latency.* Up to 10^5 keys everything fits in cache: lookups take 0.4–4 µs, non-membership proofs 0.7–2.8 µs, verifications 5–8 µs and inserts 8–19 µs. At 10^6 keys the 2.4 M-node hash map exceeds the caches and every operation pays ~20 cache misses, yet inserts still run at 24,700/s and non-membership proofs at 66,000/s on one core.
- *Memory.* The in-memory store costs about 216 bytes per node (2.44 M nodes for 10^6 keys), of which the content-address and the two child hashes are 96 bytes; the remainder is hash-map and allocator overhead that a disk-backed or arena-allocated store would remove.
- *Non-membership shapes.* About 28% of absent keys hit an empty subtree and 72% a foreign leaf; both kinds have the same size distribution.

6.4 Adversarial keys

To test the worst case, 1,000 keys were forced to share a 200-bit prefix, which the single-leaf rule cannot absorb: leaves sit at depth 211 on average (max 221). Inserts then cost 327 µs (211 hashes) and verification 198 µs – the price of hashing 200 extra levels, still bounded by 256 – but because the 200 chain siblings are all empty, the proofs contain only 10.3 non-empty siblings and average 424 bytes, essentially the same as for random keys. This confirms that the bitmask compaction, not merely the collapse rule, is what keeps proofs small, and that the 8,323-byte bound is never approached in practice.

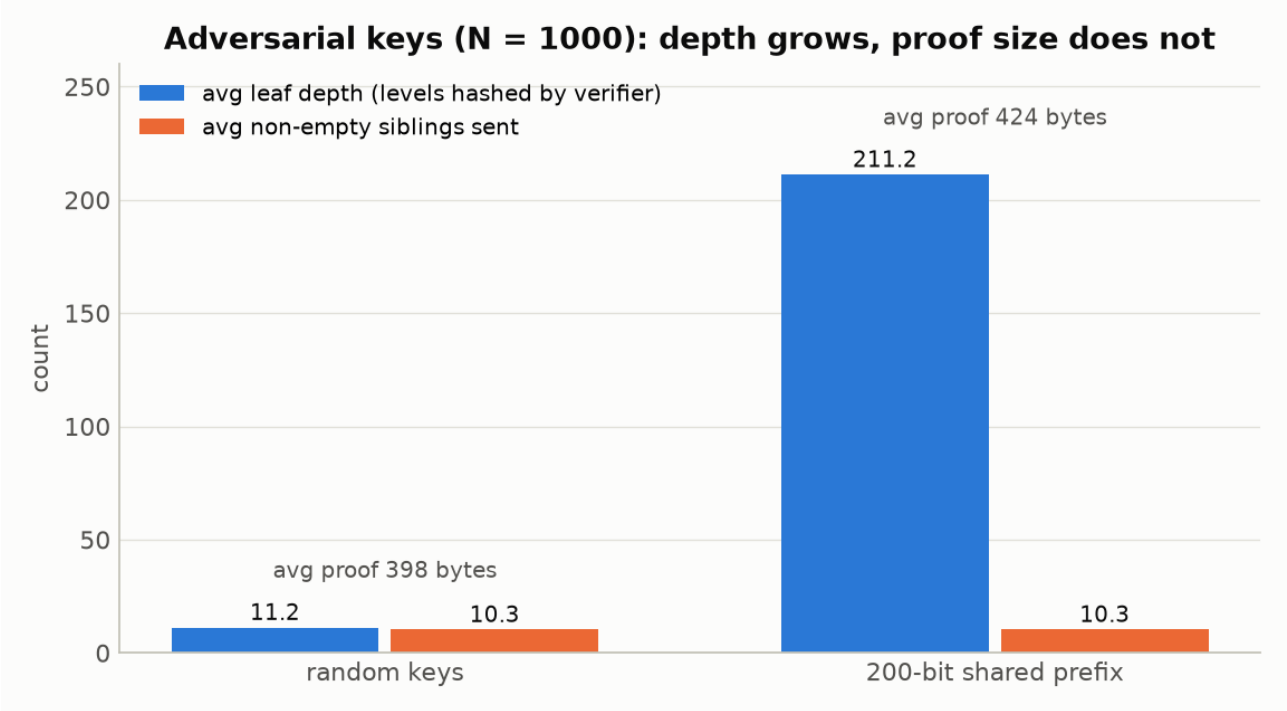


Figure 4. Adversarial versus random keys at N = 1,000: depth (hashing cost) rises twenty-fold, the number of siblings transmitted and the proof size do not.

6.5 Complexity summary

Operation	Naive 256-level SMT	This implementation (N keys, random paths)	Worst case (adversarial paths)
Lookup	256 node reads	≈ log2 N + 1 node reads	256 node reads
Insert / update / delete	256 hashes	≈ log2 N + 1 hashes, O(log N) new nodes	256 hashes
Proof generation	256 node reads	≈ log2 N + 1 node reads	256 node reads

Operation	Naive 256-level SMT	This implementation (N keys, random paths)	Worst case (adversarial paths)
Proof size	8,259 bytes (constant)	≈ 32·log2 N + 100 bytes	≤ 8,323 bytes (constant bound)
Verification	256 hashes	≈ log2 N + 1 hashes	256 hashes
Storage	O(N · 256) nodes without default-hash tricks	2N – 1 + chain nodes ≈ 2.44 N nodes	≤ 256 N nodes

7. Limitations and Future Work

- **Batch updates.** Applying a sorted batch of updates in one pass shares the upper levels of the paths and would reduce hashing roughly by the batch's shared-prefix depth; the current `batch` command applies updates one by one.
- **Disk backend.** The `NodeStore` interface is deliberately minimal so that a RocksDB/LevelDB backend can be plugged in; write-ahead logging and caching of the top levels would then be the main performance work.
- **Faster hashing.** SHA-256 with SHA-NI instructions or BLAKE3 would cut per-hash cost by 3–5×; a SNARK-friendly hash (Poseidon) would make the proofs verifiable inside zero-knowledge circuits.
- **Wider branching.** A 16-ary (nibble) tree as in the Jellyfish Merkle Tree quarters the number of levels at the cost of 15 siblings per level; with compaction the trade-off is close and depends on the storage engine.
- **Multiproofs.** Proving many keys at once can share common siblings, which matters for stateless clients that verify a whole block's accesses.
- **Concurrency.** The structure is persistent (immutable nodes), so readers never block; concurrent writers would need a path-copying commit protocol.

8. Conclusion

The project delivers a complete, tested, dependency-free C++17 implementation of an optimized 256-level Sparse Merkle Tree for state databases. By never materialising empty subtrees, floating single leaves up, and transmitting only non-empty siblings behind a presence bitmask, it reduces every operation and every proof from O(256) to O(log N) while keeping the proof size under a fixed 8,323-byte constant. Non-membership proofs – the primary requirement – are produced by the same walk as membership proofs and are sound under collision resistance of SHA-256 with RFC 6962 domain separation. Content-addressed storage makes the tree a versioned state database in which any historical root can be proven against, and garbage collection keeps the live set minimal. Measurements up to one million keys show 710-byte non-membership proofs generated in 15 µs and verified in 26 µs, with 25,000 single-core inserts per second, and the adversarial experiment shows that proof size stays flat even when depth is forced to over 200.

References

1 R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," CRYPTO 1987.

2 B. Laurie and E. Kasper, "Revocation Transparency," Google, 2012.

3 R. Dahlberg, T. Pulls, and R. Peeters, "Efficient Sparse Merkle Trees: Caching Strategies and Secure (Non-)Membership Proofs," NordSec 2016.

4 M. Bauer, "Proofs of Zero Knowledge" / Ethereum Research, "Optimizing sparse Merkle trees," ethresear.ch, 2018.

5 Celestia (LazyLedger), "smt: A Go library for a Sparse Merkle tree," github.com/celestiaorg/smt.

6 Z. Gao et al., "Jellyfish Merkle Tree," Diem Association technical report, 2021.

7 G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Appendix D (Modified Merkle Patricia Trie).

8 B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, IETF, 2013.

9 NIST, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

10 D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, Chapter 8 (collision-resistant hashing and Merkle trees).

Appendix A – Building and running

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j
./build/smt_tests                       # unit tests
./build/smt_bench --csv out.csv         # benchmark (1k..1M keys + adversarial)
bash scripts/sample_session.sh          # CLI demonstration
bash scripts/run_all.sh                 # everything above, regenerating evaluation/
python3 tests/reference_check.py        # language-independent check of the hashing spec
```

Appendix B – Sample non-membership proof (JSON)

Generated for the absent key `mallory` in the demonstration database (`evaluation/sample_proofs/proof_mallory_nonmembership.json`). The walk ended after two levels at the leaf of a different key, so the proof carries that leaf's key and value hash plus two non-empty siblings – 164 bytes in binary form versus 8,259 for the naive proof.


```
{
  "format": "smt-proof-v1",
  "type": "non-membership-leaf",
  "root": "5be5ad11a7a5732d9e6526b5df025eab306219e95dadbb40f95d2a0f359fe396",
  "user_key": "mallory",
  "key": "c0a497761b175379ed63397cc980546559faa84ca9cbeede773117c31508b6ac",
  "value_hash": "6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b",
  "other_key": "e010fd1ce1acc173e3b4835b7635f8d4600d774869102adb5cb7b5d7895649ba",
  "depth": 2,
  "bitmask": "c0000000000000000000000000000000000000000000000000000000000000000",
  "siblings": [
    "31da4f31256f30e4b4f352c676978d67b089f4006ca28289228ed2d99612ceba",
    "86a9d88f51490ff8bcff472ccd45652defed660ce4c69ee10b6f1bf8b055f15b"
  ],
  "size_bytes": 164
}
```


Appendix C – Contents of the submission archive

```
106123015_MiniProject/
  README.md                 setup and usage instructions
  CMakeLists.txt, Makefile  build system (CMake primary, make fallback)
  include/smt/*.hpp         library headers
  src/*.cpp, src/cli/       library implementation and CLI
  tests/                    unit tests, reference oracle, Python cross-check
  bench/                    benchmark harness and plot script
  scripts/                  run_all.sh, sample_session.sh, sample_batch.txt
```

```
evaluation/              test_output.txt, sample_cli_session.txt, sample_proofs/,
                         bench_output.txt, bench_results.csv, fig_*.png
report/                  Report_106123015.pdf, report.md, build_report.py
```